

Iteration

Repetitive tasks

```
name = "Ib"  
print( "Hej {}".format(name) )  
name = "Ann"  
print( "Hej {}".format(name) )  
name = "Bo"  
print( "Hej {}".format(name) )
```

The for loop

```
for name in [ "Ib", "Ann", "Bo" ]:  
    print( "Hej {}".format(name) )
```

The for loop

```
name_list = [ "Ib", "Ann", "Bo" ]
```

```
for name in name_list:  
    print( "Hej {}".format(name) )
```

The for loop

```
name_list = [ "Ib", "Ann", "Bo" ]
```

```
for name in name_list:  
    print( "Hej {}".format(name) )  
    print( "Hvordan går det?" )
```

The for loop

```
for letter in "Banana":  
    print(letter)
```

The for loop

```
for key in {"A": 3, "B": 2}:  
    print(key)
```

How to iterate over me

```
"Banana".__iter__()
```

```
["Ib", "Ann", "Bo"].__iter__()
```

```
{"A": 3, "B": 2}.__iter__()
```



Iterating over numbers

- Quick documentation: `pydoc range`
- Longer documentation: <https://docs.python.org/3/library> (kig under built-in functions”).
- Hvilke argumenter tager `range`?



The builtin `range` function

```
x = 4
```

```
for n in range(x):  
    print(n)
```

Test selv dem her

```
x = 4
```

```
for n in range(x-1, -1, -1):  
    print(n)
```

```
for n in range(1, x+1):  
    print(n)
```

```
for n in range(-1, -x-1, -1):  
    print(n)
```

Tæl til en million

```
count = 0
```

```
print(count)
```

```
for n in range(1000000):  
    count = n
```

```
print(count)
```

Prøv om du kan nå at
blinke mens programmet
kører :)

Hvad sker der her?

```
fruit = 'Banana'
```

```
length = len(fruit)
```

```
for idx in range(length):  
    character = fruit[idx]  
    print(character)
```



Hvad sker der her?

```
fruit = 'Banana'
```

```
for idx in range(len(fruit)):  
    print(fruit[idx])
```

Opgave

```
fruit = 'Banana'
```

Skriv et for-loop så i kan printe følgende:

```
0 B
1 a
2 n
3 a
4 n
5 a
```

```
fruit = 'Banana'

for idx in range(len(fruit)):
    print(fruit[idx])
```

Løsning

The for loop

```
for x in ["Ib", "Ann", "Bo"]:  
    print(x)
```

```
for x in range(1000000):  
    print(x)
```

Tanketjek

```
name_list = [ "Ib", "Ann", "Bo" ]
```

```
for name in name_list:
```

```
    name = "Mogens"
```

```
print(name_list)
```

```
for idx in range(len(name_list)):
```

```
    name_list[idx] = "Mogens"
```

```
print(name_list)
```

Break

List recap

```
names = ["Rip", "Rap", "Rup"]  
print(names[1]) # ok  
print(names[3]) # error
```

```
# re-assign existing element  
names[1] = "Mogens"  
print(names)
```

```
# add element to end of list  
names.append("Preben")  
print(names)
```



Warm-up

```
numbers = [1,2,3,4]
```

```
squares = []
```

Write two lines of code that fills the `squares` list with the square of the numbers from `numbers`.

Remember the `append` method of lists?

Løsning

Repetitive tasks

```
template = "vil {} med i Netto?"
```

```
x = "Ib"
```

```
print(template.format(x))
```

```
x = "Ann"
```

```
print(template.format(x))
```

```
x = "Bo"
```

```
print(template.format(x))
```

The for loop

```
template = "vil {} med i Netto?"
```

```
for x in ["Ib", "Ann", "Bo"]:  
    print(template.format(x))
```

for-loops

```
template = "vil {} med i {}?"
```

```
y = "Netto"
```

```
for x in ["Ib", "Ann", "Bo"]:  
    print(template.format(x, y))
```

```
y = "Zoo"
```

```
for x in ["Ib", "Ann", "Bo"]:  
    print(template.format(x, y))
```



for-loops

```
template = "vil {} med i {}?"
```

```
for y in ["Netto", "Zoo"]:  
    for x in ["Ib", "Ann", "Bo"]:  
        print(template.format(x, y))
```



for-loops

```
template = "vil {} med i {}?"  
  
for x in ["Ib", "Ann", "Bo"]:  
    for y in ["Netto", "Zoo"]:  
        print(template.format(x, y))
```



for-loops

```
template = "vil {} med i {}?"  
  
for x in ["Ib", "Ann", "Bo"]:  
    for y in ["Netto", "Zoo"]:  
        print(template.format(x, y))
```



for-loops galore

```
for i in range(3):  
    for j in range(4):  
        print(i, j)
```

Reminder

```
row = []  
for j in range(4):  
    row.append(j)
```



```
matrix = []
```

```
row = []
```

```
i = 0
```

```
for j in range(3):
```

```
    row.append(j)
```

```
matrix.append(row)
```

```
row = []
```

```
i = 1
```

```
for j in range(3):
```

```
    row.append(j)
```

```
matrix.append(row)
```

Opgave

Produce this list of lists (matrix) with 3 sublists each with 4 numbers:

```
[[0, 1, 2],  
 [0, 1, 2]]
```

Same as:

```
[[0, 1, 2], [0, 1, 2]]
```

Hint:

```
matrix = []  
for i in range(2):  
    # make an empty sublist  
    for j in range(3):  
        # fill the sublist  
    # add sublist to list of lists
```



Opgave

```
matrix = []  
for i in range(2):  
    row = []  
    for j in range(3):  
        row.append(j)  
    matrix.append(row)  
  
print(matrix)
```

Opgave

```
matrix = []  
for i in range(2):  
    row = []  
    for j in range(3):  
        row.append(i)  
    matrix.append(row)  
  
print(matrix)
```

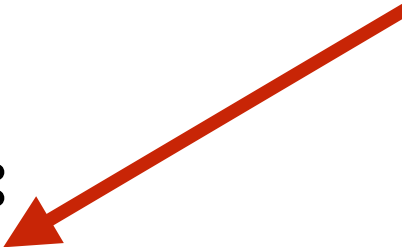
Now *i* instead of *j*



Opgave

```
matrix = []  
for i in range(2):  
    row = []  
    for j in range(3):  
        row.append(i*j)  
    matrix.append(row)  
  
print(matrix)
```

Now $i*j$



Opgave

Hvad gør funktionen?

```
def some_function(text, letter):  
    count = 0  
    for c in text:  
        if c == letter:  
            count = count + 1  
    return count
```

```
n = some_function('banana', 'a')
```